AI と一緒に、 Roblox ゲームを作る

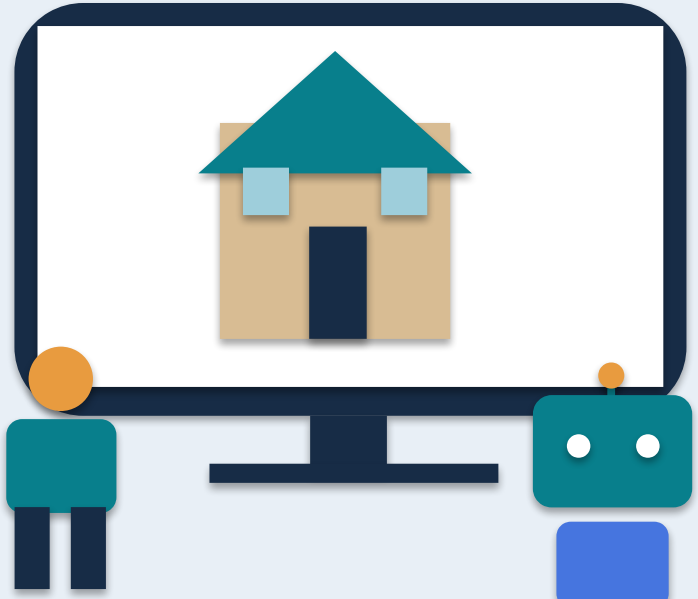
企画・設計・建物・スクリプト・テストまで、制作の進め方を 8 項目で学ぶ

人間が考え、
AI と一緒に形にする。

Claude / ChatGPT （ Codex など）と
Roblox Studio をつなぐ制作スタイル

企画　制作　確認

日本語の指示から、ゲームの世界へ



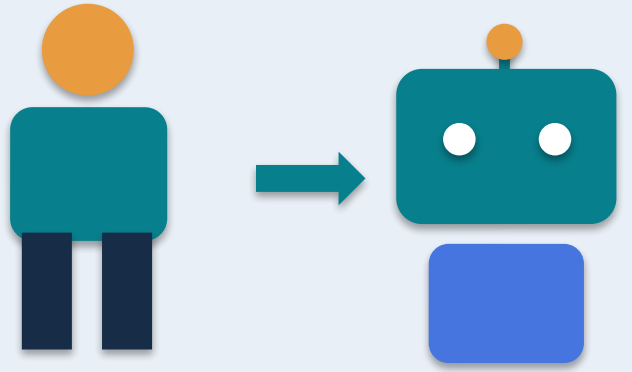
この資料の図解は制作のイメージです

AI を「制作スタッフ」として使う

人間が方向性を決め、 AI が具体的な制作を支える。

- **制作の一部を AI に任せる**
  建物・スクリプト・ UI などの作業を手伝わせます。
- **人間は「何を作るか」を決める**
  遊び方、世界観、完成の目標を言葉にします。
- **AI には「どう作るか」を相談する**
  必要な部品や処理、実装の手順を提案させます。
- **日本語で作りたい内容を伝える**
  コードをすべて覚える前から制作を始められます。
- **作ったものは人間が確認する**
  動き・見た目・面白さを確かめ、修正を指示します。

役割を分けて、一緒に作る



人間：企画・判断

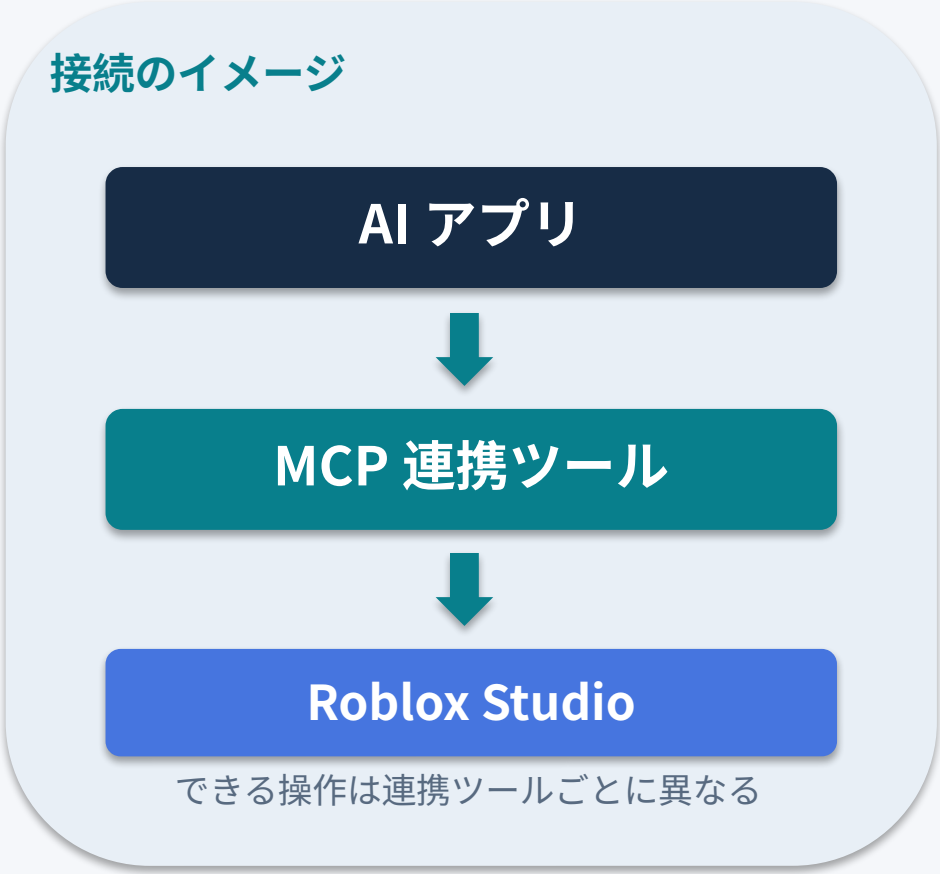
AI：提案・制作

対話 → 制作 → 確認を繰り返す

MCP は、 AI と Studio をつなぐ橋

対応する AI アプリ・連携ツールを設定して利用する。

- **MCP は「つなぐ仕組み」と考える**
  AI が外部ツールを使うための共通の仕組みです。
- **対応した連携環境を用意する**
  AI 側と Studio 側のツールを設定し、接続します。
- **ゲームの中身を AI に確認させる**
  Explorer の構造などを読めるか確かめます。
- **部品やスクリプトの作成を頼む**
  接続ツールが対応する操作を AI に実行させます。
- **最初は小さな作業で接続を確認する**
  「Part を 1 個作る」などから動作を試します。



制作を始める前に、企画書を渡す

ゲームのゴールを共有すると、 AI への指示がぶれにくくなる。

- **どんなゲームかを決める**
  農園・ FPS ・ホラー・ RPG など、ジャンルを選びます。
- **誰に遊んでもらうかを書く**
  対象年齢や、想定する PC ・スマホの操作を決めます。
- **遊びの繰り返しを整理する**
  例：植える → 育てる → 収穫する → 種を買う。
- **世界観と必要な機能を書く**
  マップ、建物、ショップ、報酬などを列挙します。
- **最初に完成させる範囲を決める**
  小さく遊べる試作品を目標にし、後で広げます。

企画書は、共通の設計図

農園ゲームの企画書

対象：小学生〜大人

目的：農園を広げる

遊び：植える・育てる・収穫

まず作る：畑 1 つと作物 1 種

「何を作るか」を先にそろえる

設計書と工程表で、作業を分ける

大きな依頼を、小さく確認できる作業へ分解する。

- **企画を実装の設計に変える**
  どんな画面・データ・機能が必要か整理させます。
- **必要な部品を一覧にする**
  Folder ・ Model ・ Script などの配置を決めます。
- **制作する順番を決める**
  例：基本操作 → マップ → ギミック → UI 。
- **一つの工程ごとに動作確認する**
  完成条件を決め、テストできてから次へ進みます。
- **人間が全体の進行を管理する**
  優先順位を判断し、企画書と進捗を更新します。

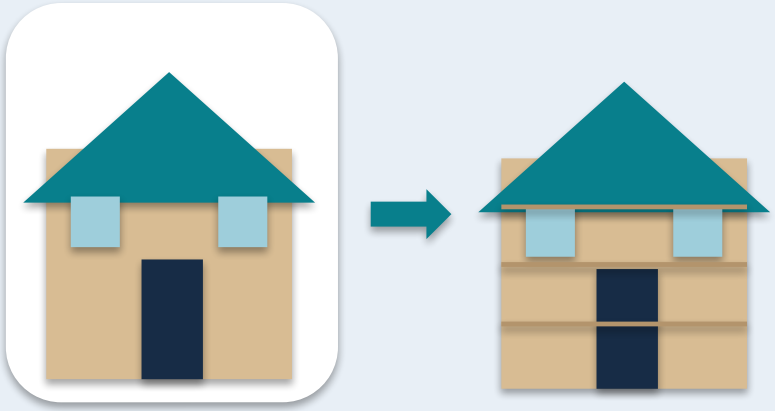


写真や参考画像から、建物を作る

見た目のイメージを共有し、 Part で形にしていく。

- **建物の参考画像を用意する**
  写真や、自分で生成したイメージ画像を使います。
- **画像を AI に見せて特徴を伝える**
  階数・屋根・窓・色など、再現したい点を示します。
- **Part を使って形を組み立てさせる**
  壁・床・屋根を分け、基本の形から作ります。
- **大きさと使い道も指定する**
  「２階建て」「中に入れる」など条件を加えます。
- **見た目と遊びやすさを調整する**
  画像と比較し、通路幅・当たり判定も確認します。

参考画像 → Part で組み立て



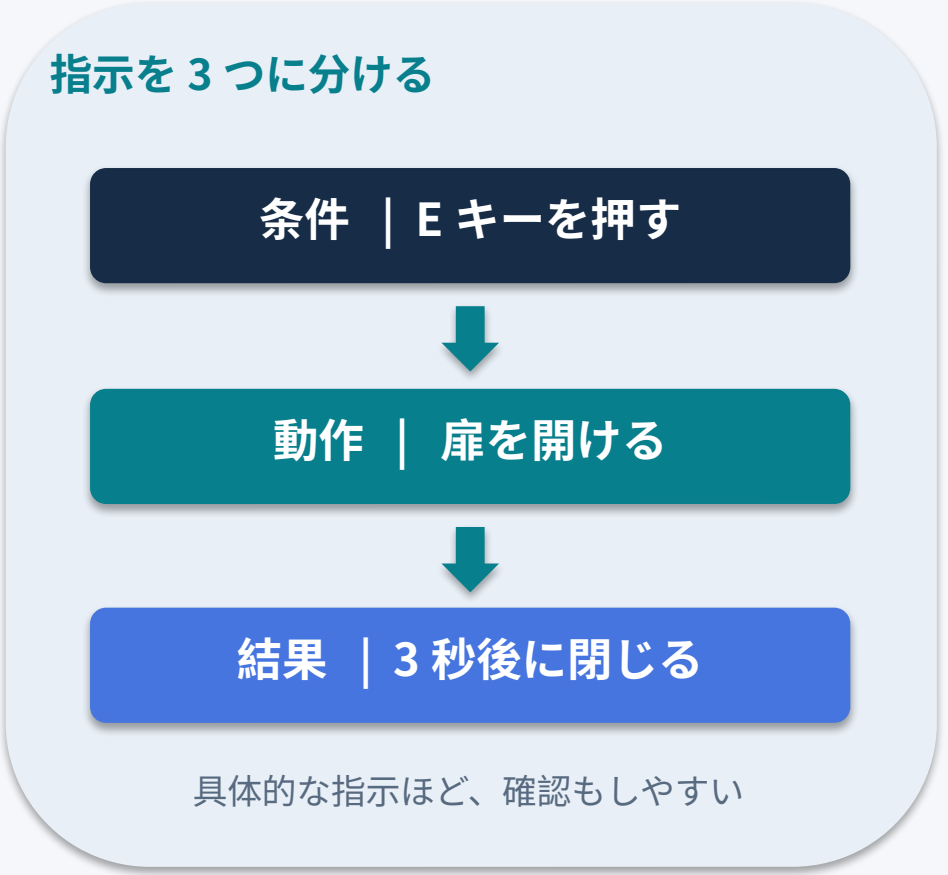
参考画像　　Part の建物

画像だけで寸法や内部は確定しない

「どう動いてほしいか」を日本語で伝える

条件・動作・結果を具体的にすると、意図が伝わりやすい。

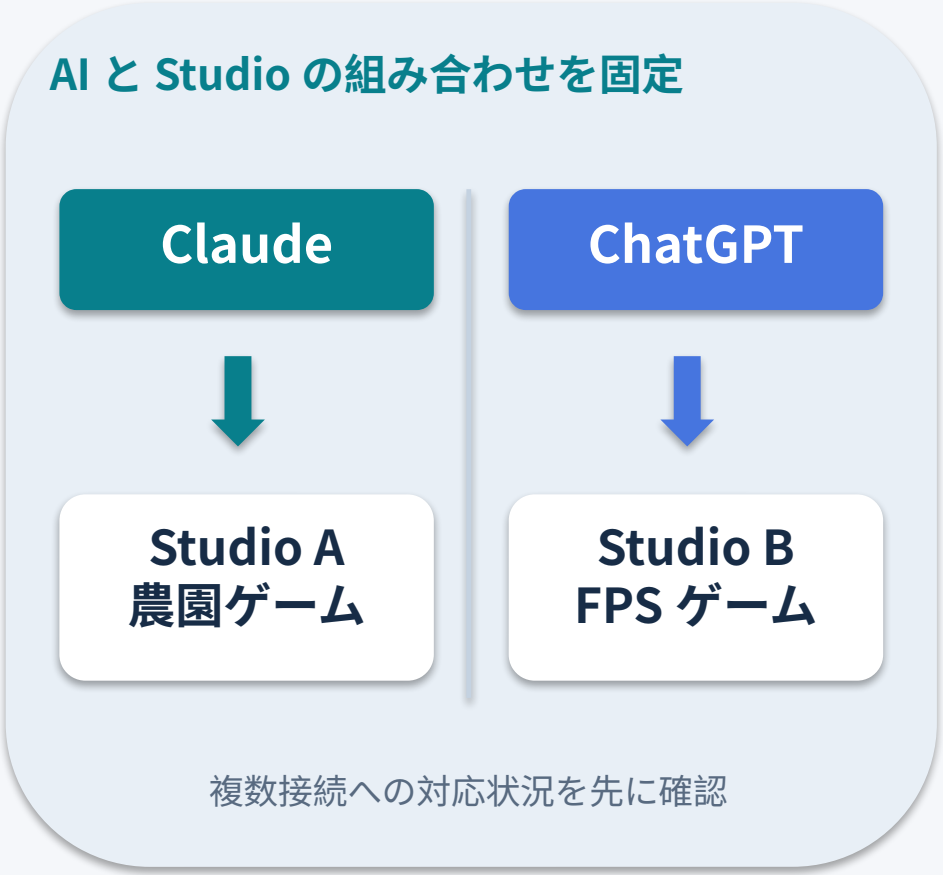
- **操作する条件を説明する**
  例:「扉に近づいて E キーを押したら」。
- **起きてほしい動作を説明する**
  例:「扉がゆっくり開き、3 秒後に閉じる」。
- **必要なルールも一緒に伝える**
  例:「持ち主だけ使える」「連打では動かない」。
- **エラーは状況とセットで渡す**
  エラー文、操作手順、期待した動きを伝えます。
- **PC ・スマホで実際に試す**
  操作ボタンや表示、他プレイヤーへの反映を確認。



1 台の PC で、 2 つの制作を進める

PC の余裕と、接続先を分けられる連携環境が前提。

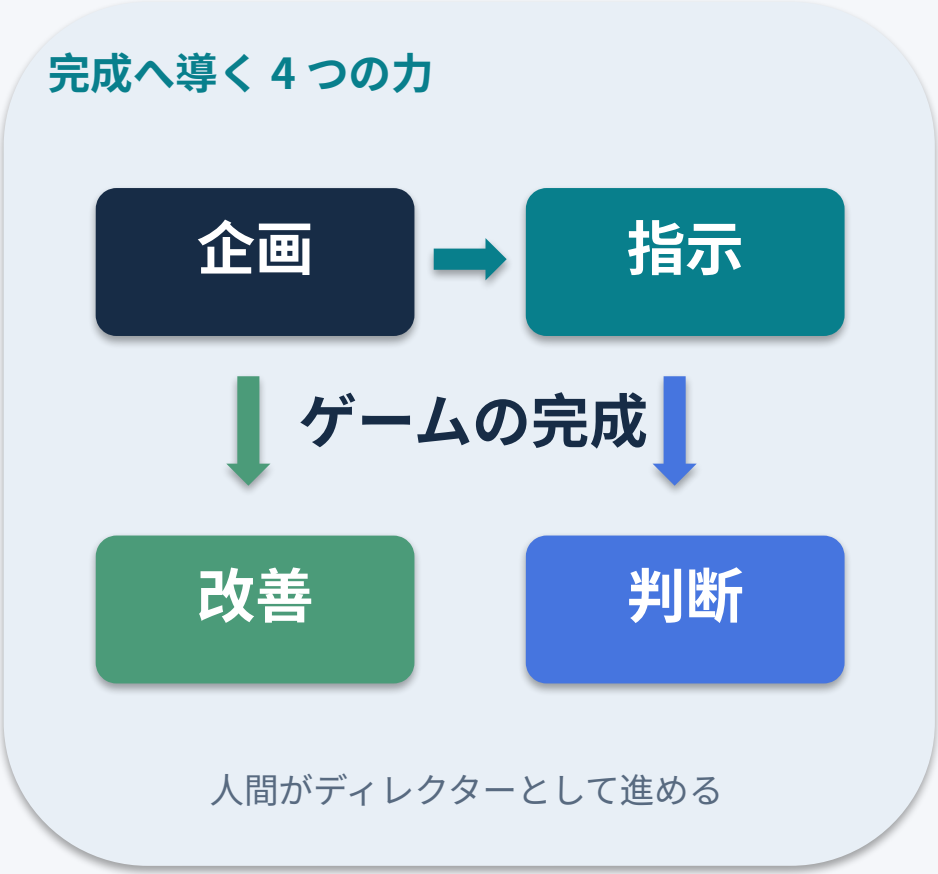
- **Studio で別々のゲームを開く**
  ゲーム A とゲーム B を、別の作業として管理します。
- **AI ごとの担当を決める**
  例： Claude は農園、 ChatGPT は FPS を担当。
- **接続先が正しいか必ず確認する**
  対応ツールで接続設定を分け、誤操作を防ぎます。
- **企画書・保存先・指示を分ける**
  名前を明確にし、別ゲームの内容を混ぜません。
- **PC 負荷を見ながら並行作業する**
  重い場合はテストを順番にし、こまめに保存します。



AI を使って、完成へ導く力を育てる

企画・指示・判断・改善を繰り返すことが大切。

- **企画する力**
  誰に、どんな楽しさを届けるのかを決めます。
- **分かりやすく指示する力**
  目的・条件・完成の基準を具体的に伝えます。
- **仕組みを理解する力**
  基本を学ぶと、 AI の提案を確認しやすくなります。
- **品質と面白さを判断する力**
  不具合、遊びにくさ、退屈な部分を見つけます。
- **最後まで改善して完成させる力**
  テストと修正を重ね、遊べるゲームに仕上げます。



最初は「小さく遊べるゲーム」を作ろう

一度に全部を依頼せず、工程ごとに作って確かめる。

01 企画
誰が、何を楽しむ?

02 設計・工程
必要な機能と順番

03 MCP 接続
接続と操作を確認

04 画像で共有
背景・建物のイメージ

05 制作
Part ・スクリプト・ UI

06 テスト・改善
実際に遊んで直す

人間が監督になり、 AI と一緒にゲームを完成へ導く。

1. ホーム（Home）タブ

制作に必要な最頻出ツールが集約された、作業の拠点となるメインタブです。

- **クリップボード操作（貼り付け・複製）**：オブジェクトのコピーやその場への複製（Duplicate / `Ctrl+D`）を即座に実行できる。
- **基本トランスフォームツール**：オブジェクトの選択（Select）、移動（Move）、拡大縮小（Scale）、回転（Rotate）の基本 4 操作を切り替えられる。
- **トランスフォームの基準切り替え**：座標軸をワールド座標基準にするか、パーツ自身のローカル座標基準にするかをワンクリックで変更できる。
- **衝突判定（Collisions）トグル**：パーツを動かす際に他のパーツをすり抜けるか、物理的にぶつかって上に乗るかを切り替えられる。
- **パーツ（Part）のクイック生成**：直方体（Block）、球（Sphere）、くさび（Wedge）、円柱（Cylinder）などの基本図形を即座に配置できる。

- **マテリアル&カラーパレット**：選択したパーツの表面材質（木材、金属、ネオン等）や色をドロップダウンから素早く適用できる。
- **グループ化（Group）と解除**：複数パーツを 1 つのモデル（Model / Folder）にまとめたり、分解したりする操作が行える。
- **アンカー（Anchor）設定**：パーツを空中に固定し、ゲーム開始時に重力で落下しないようにする最重要トグル。
- **テストプレイ（Play / Play Here / Run）**：ゲームをその場で起動して動作検証を素早く開始できる。
- **ゲーム設定（Game Settings）**：公開設定、アバターの体型比率、セキュリティ（API アクセス許可）などの全体設定画面を開く。

2. アバター（Avatar）タブ

プレイヤーや NPC のキャラクターモデル、衣服、アニメーションを設定・編集するためのタブです。

- **Rig Builder（リグビルダー）**：R6（クラシック）や R15（現代標準）の人型モデルやメッシュ

リグをワンクリックで生成・配置できる。

- **Animation Editor（アニメーションエディタ）**：生成したリグを使って、待機・歩行・攻撃などのアニメーションをキーフレーム方式で作成できる。
- **アタッチメントの可視化と調整**：アクセサリーやエフェクトをキャラクターの特定部位に固定するための基準点（Attachment）を配置・調整できる。
- **アクセサリーの試着と編集**：帽子、髪型、背中装備などのアタッチメント位置をキャラクターに合わせて微調整できる。
- **服・クラシックアパレル設定**：Shirt や Pants などのテクスチャをアバターに正しく適用・マッピングできる。
- **アニメーションのパブリッシュ**：作成したアニメーションを Roblox クラウドへアップロードし、専用のアセット ID を取得できる。
- **Face / 表情アニメーションの設定**：Dynamic Heads（動く表情）に対応したフェイシャルアニ

メーションのプレビューや設定が行える。

- **衝突グループ（Collision Groups）の連携**：キャラクター同士がすり抜ける設定や、特定プレイヤーのみ通れる壁などの判定を設定できる。
- **アバタースケール（体型）のプリセット確認**：デフォルトの身長、体幅、頭のサイズ比率を視覚的にチェックできる。
- **NPC 化・初期配置**：作成したリグにスクリプトを仕込んでゲーム内の案内人や敵キャラクターとしてワークスペースへ配置できる。

3. UI（User Interface）タブ

画面上に表示する体力ゲージ、ボタン、ショップ画面、テキストなどをデザイン・配置するためのタブです。

- **ScreenGui のワンタッチ作成**：プレイヤーの画面に 2D インターフェースを表示するためのベースキャンバスを一発で配置できる。
- **フレーム（Frame）とパネルの配置**：背景やメニューの土台となる四角形パネルを直感的に配置できる。

- **TextLabel / TextButton の生成**：文字表示用のラベルや、プレイヤーがクリック/タップできるインタラクティブなボタンを素早く追加できる。
- **ImageLabel / ImageButton の配置**：外部からアップロードした画像アイコンやイラストボタンを UI 上に配置できる。
- **UI エディタ（ビジュアル編集）**：ビューポート上で UI 要素をマウスドラッグし、直感的に位置やサイズを変更できる。
- **スケール（Scale）とオフセット（Offset）の調整**：PC、スマホ、タブレットなど画面解像度が異なってもレイアウトが崩れないようレスポンシブ設定ができる。
- **UI 角丸（UICorner）や枠線の装飾**：ボタンの角を丸めたり、外枠（UIStroke）をつけたりしてモダンなデザインへ整えられる。
- **自動レイアウト（UIListLayout / UIGridLayout）**：インベントリやアイテム一覧などを均等間隔・グリッド状に自動整列させられる。

- **ZIndex（重ね順）の視覚的管理**：複数の UI 要素が重なった際に、どれを最前面に表示するかを調整できる。
- **プレビューとデバイスエミュレーション**：各スマートフォンやコンソールの画面比率に切り替え、実機での見え方を即座に検証できる。

4. スクリプト（Script）タブ

ゲームロジックを記述する Lua コードの作成、編集、デバッグを強力にサポートするタブです。

- **スクリプト新規作成（Server / Local / Module）**：サーバー用、クライアント（端末）用、共通関数用の ModuleScript を素早く追加できる。
- **オートコンプリートとコード補助**：Roblox 固有の API やプロパティ名を自動予測表示し、タイピングミスを防ぐ。
- **ブレークポイントの設定**：コードの特定行で実行を一時停止させ、変数の値や処理の流れを 1 行ずつ追跡できる。

- **ステップ実行（Step In / Step Over / Step Out）**：デバッグ中に処理を１ステップずつ進め、意図しない挙動の原因を特定できる。
- **ウォッチ（Watch）ウィンドウの活用**：停止中に特定の変数やテーブルの中身がどう変化しているかをリアルタイム監視できる。
- **全ファイル一括検索・置換**：プロジェクト内に散らばる大量のスクリプトから、特定の文字列や関数を一括で探して修正できる。
- **構文チェックとリアルタイムエラー表示**：記述ミスや構文エラーをコード入力中に赤い波線でリアルタイムに警告してくれる。
- **コードの整形（Format Document）**：インデントの崩れをショートカット一つで自動的に綺麗なレイアウトへ整頓できる。
- **コメントアウトのクイック切り替え**：選択した複数行のコードを一括でコメント化（無効化）または解除できる。
- **バージョン履歴と差分比較**：クラウド上に自動保存された過去のスクリプト履歴と現在のコードを

比較し、巻き戻しができる。

5. モデル（Model）タブ

3D 空間における精密な配置、幾何学的な結合・くり抜き（CSG）、物理拘束を設定するタブです。

- **スナップ・増分設定（Rotate / Move Snapping）**：グリッド単位（例: 1stud 刻み）や回転角度（例: 45 度刻み）を厳密に固定して狂いなく配置できる。
- **結合（Union）による形状作成**：複数のパーツを合体させて、1 つの複雑なオリジナル形状を作成できる。
- **くり抜き（Negate / Subtract）処理**：パーツを「型」として指定し、別のパーツからその形状を削り取って穴や溝を作れる。
- **ソリッドモデリングの分離（Separate）**：Union で結合したオブジェクトをいつでも元の基本パーツ群へ分解・再編集できる。
- **コンストレイント（Constraints）の追加**：ヒンジ（蝶番）、スプリング（バネ）、ロープ（紐）などの物理ジョイントをパーツ間に配置できる。

- **モーター・スライダー機構の作成**：回転ドア、エレベーター、動く足場などの物理ギミックをコードなしで組み立てられる。
- **ピボットポイント（Pivot）の編集**：オブジェクトの回転や配置の「基準点（中心）」を自由な位置に変更できる。
- **整列ツール（Align Tool）**：複数のパーツの端面や中心を綺麗にワンクリックで一直線に揃えられる。
- **スポーン地点（SpawnLocation）の配置**：プレイヤーがゲーム開始時やリスポーン時に出現する床を即座に生成できる。
- **エフェクト（ParticleEmitter / Smoke / Fire）の追加**：炎、煙、火花、光などの環境特殊効果をパーツに直接付与できる。

6. プラグイン（Plugins）タブ

サードパーティ製拡張機能や自作ツールを管理し、Studio の機能を拡張するためのタブです。

- **Plugins Manager（プラグイン管理）**：導入済みプラグインの有効化・無効化、更新、アンインス

トールを一元管理できる。

- **Toolbox 連携（プラグインの検索・導入）**：Roblox 公式ストアから世界中の開発者が公開した便利プラグインを検索してワンクリック導入できる。
- **地形・マップ生成補助プラグインの実行**：プロシージャルな道路生成や樹木の自動散布ツールなどを起動できる。
- **3D テキスト生成プラグインの活用**：看板や案内板に使える立体的なアルファベット・数字メッシュを自動作成できる。
- **アニメーションカーブ・外部リグ変換ツール**：Blender などから取り込んだ複雑なボーン構成を Roblox 用に適合させるプラグインを動かせる。
- **UI グラデーション・デザイン補助ツール**：リッチな UI デザインやアイコン配置をサポートする専用エディタを呼び出せる。
- **ライティング・ポストエフェクト一括設定**：夕焼け、夜景、サイバーパンク風など、光のパラメータをワンクリックで最適化するツールを使える。

- **自作プラグインの保存（Save as Local Plugin）**：自分が書いたスクリプトをローカルプラグイン化し、すべてのプロジェクトで使い回せる。
- **セキュリティ・スクリプトスキャン**：外部アセットに含まれる不正なウイルスコードやバックドアスクリプトを検出・除去するプラグインを実行できる。
- **外部通信・ブリッジ連携の起動**：Blender 連携や MCP、外部ローカルサーバーと Studio を通信させるためのプラグインスクリプトを常駐・実行させる。

1. ファイル（File）メニュー

プロジェクト全体の保存、公開、外部アセットの読み書き、システム設定を一元管理するメニューです。

- **新規作成（New）**：新しいテンプレート（Baseplate や各種テーママップ）から白紙のプロジェクトを立ち上げる。
- **ローカルファイルを開く（Open from File）**：PC のストレージに保存されている `.rbxl`（プロジェクトファイル）を開く。
- **クラウドから開く（Open from Roblox）**：自分のアカウントや所属グループで管理しているクラウド上のゲームを選択して開く。
- **ローカルへ保存（Save to File / Save to File As）**：現在の作業データを PC 内にローカルファイル（`.rbxl`）として名前を付けて保存する。
- **Roblox へ保存（Save to Roblox）**：クラウド上のデータに対して下書き保存（ドラフト保存）を行い、未公開のまま最新状態を保持する。

- **Roblox へ公開（Publish to Roblox / Publish to Roblox As）**：実際の一般プレイヤーがアクセスする本番環境へ更新内容を反映（デプロイ）する。
- **場所（Place）の追加・管理**：1 つの「体験（Game）」の中に、ロビーや別ステージなど複数のマップ（Place）を追加・紐付ける。
- **ゲーム設定（Game Settings）**：タイトル、アイコン、収益化設定、アバター仕様、アクセス権限などを設定するダイアログを開く。
- **Studio 設定（Studio Settings）**：エディタのダークモード切り替え、レンダリング品質、カメラ感度、ディレクトリ設定などを変更する。
- **最新アセット・最近使ったファイルの履歴**：直近で編集したローカルプロジェクトやクラウドファイルを履歴から素早く再開する。

2. 編集（Edit）メニュー

ワークスペース上のオブジェクトやスクリプトに対する基本的な編集・選択補助を行うメニューです。

- **元に戻す（Undo / `Ctrl+Z`）**：直前に行ったオブジェクトの移動、削除、プロパティ変更などの操作を１段階巻き戻す。
- **やり直し（Redo / `Ctrl+Y`）**：取り消した操作を再び適用して最新の状態へ戻す。
- **切り取り（Cut / `Ctrl+X`）**：選択中のオブジェクトやテキストをクリップボードに退避させ、その場から削除する。
- **コピー（Copy / `Ctrl+C`）**：選択中のオブジェクトの形状・設定をクリップボードに記憶させる。
- **貼り付け（Paste / `Ctrl+V`）**：コピーしたオブジェクトをワークスペースの視界内や対象親階層の中へ配置する。
- **その場に貼り付け（Paste Into）**：選択している特定のフォルダやモデルの「子階層」として直接貼り付ける。
- **複製（Duplicate / `Ctrl+D`）**：選択中のオブジェクトを全く同じ座標・プロパティでその場に複製する。

- **削除（Delete）**：選択中のオブジェクトやスクリプトをワークスペースから完全に破棄する。
- **すべて選択（Select All / `Ctrl+A`）**：現在開いている階層内の全オブジェクト、またはスクリプト内の全テキストを一括選択する。
- **選択の反転（Invert Selection）**：現在選択しているもの以外のすべてのオブジェクトを選択状態へ反転させる。

3. 表示（View）メニュー

Studio の画面レイアウトをカスタマイズし、必要な各種パネルやウィンドウの表示/非表示を切り替えるメニューです。

- **Explorer（エクスプローラー）の開閉**：ゲーム内の全オブジェクト、スクリプト、ライティングがツリー状に並ぶ最重要パネルを表示する。
- **Properties（プロパティ）の開閉**：選択したオブジェクトの色、サイズ、マテリアル、数値などを細かく編集するパネルを表示する。

- **Toolbox（ツールボックス）**：公式・コミュニティが公開したモデル、画像、オーディオ、メッシュを検索・配置できるストアを表示する。
- **Output（出力ウィンドウ）**：スクリプトの実行ログ（`print` 出力）やエラー通知、警告をリアルタイムで確認できるコンソールを表示する。
- **Asset Manager（アセットマネージャー）**：ゲームにアップロード済みのメッシュ、画像、サウンド、別プレイスを一括管理するウィンドウを開く。
- **Command Bar（コマンドバー）**：Studio 実行中に Lua スクリプトを 1 行直接打ち込み、即座にコマンドテストやバッチ処理を実行するバーを表示する。
- **Script Analysis（スクリプト分析）**：コード内の警告、非推奨 API の使用、潜在的な構文エラーを自動検知して一覧化するパネルを開く。
- **Terrain Editor（地形エディタ）**：山、海、洞窟、草地などのボクセル地形を生成・彫刻・ペイントするための専用ツールを開く。

- **Device（デバイスエミュレータ）**：スマホ、タブレット、家庭用ゲーム機など、各解像度での UI や画面の見え方をシミュレートする画面を開く。
- **ワイヤーフレーム・グリッド表示トグル**：パーツのポリゴン境界線（Wireframe）やグリッド線の表示・非表示を切り替えて視認性を調整する。

4. プラグイン（Plugins）メニュー

メインリボンとは別に、導入した拡張機能の個別設定や権限、外部接続の管理を行うメニューです。

- **プラグインの権限管理（Plugin Permissions）**：外部サーバーへの通信（HTTP リクエスト）やスクリプト書き込みの権限を個別に許可・拒否する。
- **インストール済みプラグイン一覧の確認**：現在 Studio に読み込まれているすべての拡張ツールの稼働状態を確認する。
- **プラグインの更新チェック**：マーケットプレイスから導入したプラグインに新しいバージョンがあるかをスキャンしてアップデートする。

- **ローカルプラグインフォルダを開く**：自作したLuaスクリプトをプラグイン化して配置するためのPC内フォルダを直接開く。
- **選択中スクリプトのプラグイン化保存**：エクスプローラーで組んだツールを「ローカルプラグインとして保存」し、他プロジェクトでも使えるようにする。
- **ショートカットの割り当て設定**：頻繁に使うお気に入りプラグインの起動を、キーボードの任意のキーにバインドする。
- **外部連携用ブリッジプラグインの監視**：BlenderやMCPサーバーと通信するバックグラウンドツールの接続ログや動作状態を確認する。
- **プラグインによるエラーログのフィルタリング**：プラグイン起因の警告やエラーが出力ウィンドウを埋め尽くさないよう切り分ける。
- **サードパーティ製ウィジェットのドッキング制御**：画面上にポップアップしたプラグイン独自の操作パネルを用紙や左右の枠へドッキングする。

- **不要プラグインのアンインストール**：使わなくなった拡張機能を Studio 上から安全に削除・整理する。

5. テスト（Test）メニュー

マルチプレイの検証、通信遅延のシミュレーション、デバッグプレイを精密に行うメニューです。

- **ローカルサーバー起動（Start Local Server）**：擬似的なマルチプレイ環境として「サーバー１台＋ 複数クライアント画面」を同時に立ち上げて同期を検証する。
- **プレイヤー人数の指定テスト**：ドロップダウンで「2 Players」「3 Players」などを選び、複数人が入室した際の挙動を一発でテストする。
- **Play（通常テストプレイ）**：現在のカメラ位置に関わらず、設定された SpawnLocation（初期地点）からキャラクターを動かしてプレイする。
- **Play Here（現在地からテスト）**：Studio 内でカメラを向けている「目の前の座標」に直接キャラクターをスポーンさせて即座に動作確認する。

- **Run（物理・スクリプト単体実行）**：プレイヤーのアバターを出現させず、重力・物理演算・サーバー側スクリプトだけをその場で稼働させる。
- **ミュート / オーディオ確認**：テストプレイ中の BGM や効果音（Sound）を即座にミュートして、静かな環境でデバッグを行う。
- **ネットワークシミュレーション（遅延エミュレーション）**：わざと Ping（通信遅延）やパケットロスを発生させ、回線の悪いプレイヤーの同期ズレを検証する。
- **エミュレーション設定（言語・地域）**：多言語対応のゲームにおいて、英語圏・アジア圏など異なる言語環境でどう表示されるかをテストする。
- **スクリプト一時停止（Pause）**：実行中のゲームをコマ送り・静止させ、物理演算の崩壊やコードの挙動をスローペースで観察する。
- **Stop（テスト終了 / `Shift+F5`）**：実行中のテストプレイを中断し、元の編集モード画面へ安全に復帰する。

6. ウィンドウ（Window）メニュー

マルチモニター環境や画面の乱れに対応し、エディタ全体の UI パネル配置を整理・リセットするメニューです。

- **レイアウトの初期化（Reset Layout）**：移動や誤操作でバラバラになってしまったエクスプローラーやプロパティの配置を工場出荷状態に戻す。
- **フローティングウィンドウのドッキング復元**：画面外に飛び出してしまったパネルをドラッグし、Studio の枠内へ再度マグネット吸着させる。
- **全画面表示（Full Screen / F11）**：OS のタスクバーやウィンドウ枠を隠し、Studio 全体をモニターいっぱいに最大化表示する。
- **タブの切り替えと分割**：開いている複数のスクリプトやビューポートのタブを並べ替え、左右・上下に分割表示する。
- **ピン留め・オートハイド（自動非表示）**：使わないパネルを画面端のアイコンへ小さく畳み込み、ビューポートの作業領域を広く確保する。

- **マルチモニター分離**：スクリプトエディタや出力ウィンドウだけを別ウィンドウとして切り離し、サブモニターへ移動させる。
- **アクティブウィンドウのフォーカス移動**：キー操作でエクスプローラーからコードエディタ、ビューポートへと選択枠を素早く移動させる。
- **最小化・最大化・閉じるのウィンドウ制御**：Studio 全体の作業領域の縮小・展開をメニューから素早く行う。
- **開いているドキュメント一覧の管理**：裏で開きっぱなしになっている大量のスクリプトタブを一覧化し、不要なものを一括で閉じる。
- **ビューポートの縦横比固定**：特定の解像度（16:9 やウルトラワイド等）で画面枠を固定し、視覚的な崩れがないか確認する。

7. ヘルプ（Help）メニュー

公式ドキュメントへのアクセス、開発コミュニティへの参加、不具合報告を行うメニューです。

- **Roblox Creator Hub（公式ドキュメント）への直通**：関数、イベント、クラスの公式 API リファレンスサイトをブラウザで開く。
- **チュートリアルとガイド**：初心者向けのステップバイステップ開発ガイドやベストプラクティス集へアクセスする。
- **Creator DevForum（開発者フォーラム）**：世界中のクリエイターがバグ報告、質問、技術共有を行っている公式コミュニティを開く。
- **ショートカットキー一覧の確認（Shortcuts）**：作業効率を飛躍的に高める Studio 内のキーボードショートカット割り当て一覧を確認する。
- **新機能・リリースノート（Release Notes）**：Studio のバージョンアップデートで何が追加・修正されたかの公式更新履歴を確認する。
- **バグ報告・フィードバック送信**：エディタのクラッシュや予期せぬ不具合を Roblox エンジニアチームへ直接レポートする。

- **Developer Status（サーバーステータス）**：Roblox のクラウドサーバーや API がダウン・障害を起こしていないかをリアルタイム確認する。
- **ライセンスおよび権利表記**：Studio 内で使用されているオープンソースソフトウェアのライセンス情報を確認する。
- **Roblox Studio のバージョン情報（About Roblox Studio）**：現在インストールされている Studio のビルド番号やバージョンをチェックする。
- **公式アナウンス・ロードマップの確認**：今後実装が予定されている大型機能やエンジンのアップデート予定情報をチェックする。

1. Workspace（ワークスペース）

プレイヤーの目に見える「3D 世界そのもの」を管理する一番基本の場所です。画面に出したい物体はすべてここに置きます。

- **目に見える 3D 物体の置き場所**：ブロック、建物、地面、木など、プレイヤーが触れたり見たりできるものは原則ここに配置される。
- **初期の床（Baseplate）の格納**：プロジェクト作成時に最初からある地面の板（Baseplate）もこの直下に入っている。
- **カメラ（Camera）の管理**：ゲームプレイ中にプレイヤーの視点を映し出しているカメラオブジェクトが自動で管理されている。
- **重力や物理演算が働く空間**：アンカー（固定）されていないパーツは、ここに置くことで重力によって自然落下する。
- **地形データ（Terrain）の保持**：地形ツールで盛った山や掘った川などのボクセル地形データがここに保存される。
- **出現したプレイヤーキャラクターの移動先**：ゲームが始まると、参加したプレイヤーの 3D モデル（体）が一時的にここへ配置される。

- **NPC や敵キャラクターの配置先**：街の村人やモンスターなど、ワールド内を歩き回る人型モデルもここに置く。
- **落下判定ライン（FallenPartsDestroyHeight）**：パーツやキャラクターが奈落の底へ落下した際に自動消滅する高さの基準を持つ。
- **フォルダによる整頓が推奨**：パーツが増えると散らかりやすいため、「Folder」を作って建物別・エリア別にまとめるのが基本。
- **スクリプトからの最頻出アクセス**：Lua コードを書く際、workspace.Part や game.Workspace として一番多く触れることになる。

2. Players（プレイヤーズ）

ゲームに参加している「プレイヤー自身のアカウント情報や状態」を一元管理するサービスです。

- **参加中プレイヤーの一覧リスト**：現在サーバーに接続しているユーザーがアカウント名ごとのオブジェクトとして自動で並ぶ。
- **3D モデルではなく「データ」の置き場**：体（アバター）そのものは Workspace にあり、ここにはユーザー ID や設定などのデータが入る。
- **ゲーム開始・退出イベントの検知**：プレイヤーが入室した時（PlayerAdded）や退出した時

（PlayerRemoving）を検知するスクリプトで必ず使う。

- **リーダーボード（戦績表・所持金表示）の作成先**：画面右上に出るスコアやコイン数を管理する「leaderstats」フォルダを各プレイヤー内に生成する。
- **ユーザー固有情報の取得**：プレイヤーの Roblox ID、表示名（DisplayName）、フレンド関係などをスクリプトから参照できる。
- **最大参加人数の基準**：サーバーに同時に何人まで入れるかといった人数管理の根幹を担っている。
- **個別チャットや権限の管理**：特定のアカウントだけに管理者権限を与えたり、個別にアイテムを渡したりする際の宛先となる。
- **テストプレイ時の確認場所**：Studio でテストを開始した際、擬似的な「Player1」がここに生成されているかを確認できる。
- **インベントリ（Backpack）の親**：プレイヤーが持っているアイテム袋（Backpack）は、この中の各プレイヤーデータにぶら下がる。
- **直接パーツを入れる場所ではない**：初心者が見落としがちな点として、ブロックやモデルを手動で直接ここへ配置することはない。

3. Lighting（ライティング）

ゲーム世界全体の「光、影、空、空気感（天候・雰囲気）」を司る場所です。

- **太陽・月の位置と時刻変更**：TimeOfDay プロパティの数値を弄るだけで、昼・夕方・夜・深夜を自由自在に変更できる。
- **空模様（Sky）のカスタマイズ**：中に「Sky」オブジェクトを入れることで、青空、星空、宇宙、不気味な赤い空などに変更できる。
- **影の濃さとレンダリング品質**：リアルな影を落とす「ShadowMap」や「Future」など、グラフィックの描画方式を切り替えられる。
- **霧（Atmosphere / Fog）の演出**：朝霧、遠くが霞む空気感、視界が遮られる深い霧などを発生させて奥行き感を出せる。
- **ブルーム（Bloom）効果の追加**：光がぼんやり発光して見えるエフェクトを追加し、ネオンや太陽をまぶしく表現できる。
- **カラー補正（ColorCorrection）**：画面全体の明るさ、コントラスト、色合いを調整し、レトロ風やホラー風の色調を作れる。
- **被写界深度（DepthOfField）でぼかし表現**：遠くの景色や近くの物体を一眼レフカメラのようにピント調整してぼかすことができる。

- **太陽光線（SunRays）の追加**：木々の隙間や建物の間から光の筋（サンレイ）が差し込むリッチな表現ができる。
- **光の強さ（Brightness）と環境光**：ワールド全体の基本照度を上げ下げし、洞窟のような真っ暗な空間も作れる。
- **時計を進める演出の実装**：スクリプトから時間を徐々に進めることで、自然な「昼夜の移り変わりサイクル」を表現できる。

4. ReplicatedStorage（レプリケーテッド・ストレージ）

サーバーと全プレイヤー（画面側）の「双方が自由に見たり使ったりできる共有の倉庫」です。

- **共有データ・アセットの置き場所**：ゲーム開始時には画面に出さないが、途中で出現させたい敵モデルや弾丸などを保管しておく。
- **サーバーと画面の架け橋（RemoteEvent）**：「ボタンを押した」「攻撃が当たった」などの合図をやり取りする通信オブジェクトを置く最重要ポイント。
- **双方向通信（RemoteFunction）の配置**：「今の所持金はいくら？」と画面からサーバーへ問い合わせて結果を受け取る仕組みを置く。

- **共通のプログラム（ModuleScript）の保存**：計算式や共通ルールなど、サーバー側でもプレイヤー側でも使いたい関数コードを保管する。
- **自動レプリケーション（自動複製同期）**：ここに置いたデータは、ゲーム開始時にすべての参加プレイヤーの端末へ自動ダウンロードされる。
- **Workspace を重くしないための待機部屋**：最初から 3D 世界に大量のモデルを置くと重くなるため、必要な瞬間までここにしまっておく。
- **アセットのクローン（複製）生成元**：銃を撃った瞬間に、ここに保管してある「弾のパーツ」をスクリプトで Workspace へコピーして飛ばす。
- **効果音・BGM・画像データの保管先**：UI 演出やゲーム中に鳴らす共通のサウンド素材を入れておく場所としても最適。
- **誰でも中身が見える（セキュリティ注意）**：プレイヤー側の端末にもデータが送られるため、見られて困る極秘コードやパスワードは置いてはいけない。
- **ゲーム制作中盤以降の必須フォルダ**：本格的なギミックやマルチプレイ通信を作ろうとすると、必ず毎日触ることになるフォルダ。

5. ServerScriptService（サーバー・スクリプト・サービス）

ゲームの重要ルールや裏側の処理を実行する「サーバー専用スクリプトの作業部屋」です。

- **サーバー用 Script（通常スクリプト）の定位置**：ゲーム全体の進行、セーブ処理、当たり判定などの大元プログラムを置く場所。
- **プレイヤー（悪意あるユーザー）から中身が見えない**：コードが各プレイヤーの PC やスマホに送信されないため、チート改ざんを完全に防げる。
- **セーブデータ機能（DataStore）の記述先**：プレイヤーのレベルやコイン数をクラウドに保存・読み込みするスクリプトはここに置く。
- **勝敗判定やゲームサイクルの制御**：「試合開始 → 制限時間カウント → 勝者判定 → ロビーへ戻す」といった全体ループを処理する。
- **課金・ゲーム内課金の処理**：Robux を使ったアイテム購入やゲームパスが正常に買われたかを安全に認証・付与する。
- **チャットコマンドの実装**：「/spawn」などの文字入力を受け取って特別な処理を行う管理者コマンドを仕込める。

- **NPC の知能（AI ロジック）の制御**：敵がプレイヤーを見つけて追いかけたり攻撃したりする基本思考コードを安全に実行する。
- **画面（LocalScript）はここでは動かない**：画面のボタンを光らせたりカメラを動かしたりする「端末側」のコードを置いても機能しない。
- **パーツにスクリプトを埋め込まない現代の標準**：パーツの中にスクリプトを１個ずつ入れる旧方式ではなく、ここにまとめて一元管理するのがプロの書き方。
- **ゲームの「心臓部・頭脳」**：ゲームが正しく公平に動作し続けるための最も厳格で安全なプログラム格納庫。

6. ServerStorage（サーバー・ストレージ）

サーバー側だけがアクセスできる、プレイヤーには絶対に覗かれない「完全秘密の金庫」です。

- **プレイヤー端末へ送信されない保管庫**：ReplicatedStorage と違い、データがユーザーの端末に一切ダウンロードされない完全非公開エリア。
- **チート・データぶっこ抜き防止**：未公開のマップ、レア武器のモデル、極秘アセットなどをクラッカーから保護できる。
- **サーバー側スクリプト専用の参照元**：ServerScriptService にあるスクリプトからのみ、中のデータを取り出して利用できる。

- **本番用マップのストック場所**：ミニゲーム集などで、出番が来るまでステージの全パーツをここに隠して待機させておく。
- **強力な武器・報酬アイテムの保管**：プレイヤーが条件を満たした瞬間にだけ、ここからインベントリへクローンして手渡す。
- **サーバー専用の共通コード（ModuleScript）置き場**：プレイヤー側に見せる必要のない、内部計算専用のモジュールを保管する。
- **通信負荷（ロード時間）の軽減**：全データを初回読み込みさせないため、プレイヤーがゲームに参加する際のロード時間を短縮できる。
- **画面側（LocalScript）からはアクセス不能**：プレイヤー側のコードからここを参照しようとするとエラーになり中身は見えない。
- **「ReplicatedStorage」との使い分けが重要**：みんなに配るものは Replicated、サーバーだけで隠し持つものは ServerStorage と覚える。
- **大規模・商業レベルゲームの必須設計**：ゲームのセキュリティとパフォーマンスを高めるために欠かせないバックヤード。

7. StarterGui（スターター・ジーユーアイ）

体力ゲージ、所持金表示、ショップボタンなど「2D の画面 UI」を開発・デザインする場所です。

- **2D インターフェースの制作拠点**：画面の四隅に置くボタン、文字、HP バー、インベントリ枠などをここに作成する。
- **ScreenGui を最初に追加する**：パーツを置く前に、まず「ScreenGui（2D キャンバス）」を追加し、その中にボタンや枠を入れていく。
- **スポーン時の自動配布システム**：名前に「Starter」とある通り、ゲーム開始（リスポーン）時に中身がプレイヤー画面へ自動コピーされる。
- **クリック・タップ操作の受付**：画面のボタンが押された時のアニメーションや画面切り替えを直感的に配置できる。
- **端末ごとの画面サイズ対応**：スマホ、タブレット、PC など画面比率が変わっても崩れないようレスポンシブ配置を組む場所。
- **チャット欄や標準 UI のカスタマイズ**：Roblox 標準のチャットウィンドウやヘルスバーの有効/無効をスクリプトで切り替えられる。
- **リスポーン時の UI リセット設定**：倒れて復活した際にメニューを開いたままにするか、初期状態に戻すか（ResetOnSpawn）を設定できる。
- **ポップアップ画面の配置**：「ステージクリア！」「レベルアップ！」などのダイアログ画面をデザインして待機させておく。

- **LocalScript との組み合わせ**：ボタンが押されたら「ショップを開く」といった演出アニメーションは、ここの中に LocalScript を書いて動かす。
- **Studio 上で直接見ながら編集可能**：ここに GUI を配置すると、Studio の作業画面上にそのままプレビューが表示され直感的に作業できる。

8. StarterPack（スターター・パック）

ゲーム開始時やリスポーン時に、プレイヤーの持ち物袋（アイテムスロット）へ「自動で配る道具」を入れる場所です。

- **初期装備アイテムの置き場所**：最初から持たせたい剣、銃、懐中電灯、回復ポーションなどをここに入れておく。
- **「Tool」オブジェクトを格納する**：ただのパーツではなく、プレイヤーが手に持てる特殊な「Tool」オブジェクトを配置するルール。
- **画面下スロット（ホットバー）への自動登録**：ここに入れたアイテムは、ゲーム開始時に画面下の 1〜9 番のスロットへ自動で並ぶ。
- **リスポーンごとの自動再配布**：プレイヤーが倒れて復活するたびに、新品のアイテム一式が再び手持ち袋へ配られる。

- **手持ちアクションの組み込み**：クリックした時に「剣を振る」「銃を撃つ」といった動作スクリプトを Tool の中に仕込んでおく。
- **持ち物の持ち手位置（Grip）の調整**：アバターが道具を握る角度や位置がおかしい場合、Tool のプロパティから持ち手を微調整できる。
- **不要な場合は空っぽで OK**：素手でアスレチック（Obby）を遊ばせるゲームなど、道具が不要な場合は何も入れなくてよい。
- **ドロップ（投げ捨て）の許可設定**：バックスペースキーなどでプレイヤーがその道具を地面に捨てられるかどうかを設定できる。
- **途中入手アイテムとの違い**：ゲーム中の宝箱やショップで買うアイテムはここではなく、ServerStorage などから直接手渡す。
- **初心者が最も簡単に武器を持たせられる場所**：Toolbox から持ってきた武器をここへドラッグ＆ドロップするだけで、即座に手持ち武器になる。

9. StarterPlayer（スターター・プレイヤー）

参加するプレイヤーの「初期能力値（足の速さ・ジャンプ力）や共通操作スクリプト」を定義する設定フォルダです。

- **移動速度（WalkSpeed）の初期値設定**：デフォルトの 16 から数値を変更し、鈍足なホラーゲームや超高速ダッシュゲームを作れる。
- **ジャンプ力（JumpPower / JumpHeight）の変更**：高くジャンプさせたり、ジャンプを完全に禁止（0 に設定）したりできる。
- **カメラ視点の固定・制限**：一人称視点（FPS）に強制固定したり、逆にカメラを大きく引いた見下ろし視点に固定できる。
- **キャラクター操作スクリプト（StarterCharacterScripts）**：プレイヤーの「体」が生成されるたびに、そのキャラクターの中に自動で注入されるスクリプトを置くサブフォルダ。
- **プレイヤー個別スクリプト（StarterPlayerScripts）**：キャラクターの生死に関係なく、入室から退出まで「端末側でずっと動き続ける」スクリプトを置くサブフォルダ。
- **アバターの当たり判定設定**：プレイヤー同士が触れたときに押し合えるか、すり抜けるかの基本挙動をコントロールできる。
- **カスタムアニメーションの適用**：Roblox 標準の歩き・走り・ジャンプモーションを、オリジナルのアニメーションへ丸ごと差し替えられる。

- **落下ダメージや体力自然回復の調整**：体力が自動で回復する仕組みのオン/オフや、落下時の挙動スクリプトを差し替える場所。
- **名前表示・ヘルスバーの非表示**：頭上に浮かぶユーザー名や緑色の HP バーを隠して、没入感を高める設定ができる。
- **「プレイヤーの土台ルール」を決める最上流**：ゲーム全体における「プレイヤーはどういう身体能力で、どうカメラが追従するか」の根本ルールを決める司令塔。

1. プロパティの基本ルール（はじめの一歩）

まずは「プロパティ画面とは何か」「どうやって使うのか」の超基本です。

- **物のプロフィール画面**：ブロックなどを１つカチッとクリックすると、その物の詳しい特徴がズラリと表示される。
- **画面の右下あたりにある**：もし見当たらないときは、画面の上の「表示（View）」から呼び戻せる。
- **選ばないと真っ白**：何も選んでいない状態では空っぽになり、物を選んだ瞬間に中身が現れる。
- **数字やチェックを入れるだけ**：難しいプログラムを書かなくても、数字を変えるだけで物が変化する。
- **変化がその場ですぐ見える**：数字を変えたり色を選んだりした瞬間、目の前のブロックの見た目が変わる。
- **何を選ぶかで中身が変わる**：ブロックを選んだ時と、光や文字を選んだ時では並ぶ項目が自動で切り替わる。
- **間違えてもすぐ戻せる**：おかしな数値を入れて形が崩れても、キーボードの「元に戻す（Ctrl+Z）」で直せる。

- **矢印を押して開け閉めできる**：項目が多すぎるときは、横にある小さな矢印を押すとグループごとに畳める。
- **アルファベット順に並べ替え可能**：上のボタンを押すと、項目の並びを「種類ごと」から「ABC順」に切り替えられる。
- **探したいときは上の検索窓を使う**：項目が多くて見つからないときは、上の小さな虫眼鏡の枠に名前を打ち込めば一発で見つかる。

2. 見た目・デザイン（見た目の印象を変える）

ブロックの色や材質など、パッと見の質感を決める項目です。

- **Color（カラー）**：絵の具パレットのような画面を開いて、好きな色を自由に選んで塗れる。
- **BrickColor（基本の色）**：昔ながらのブロックのような、決まった名前のついた色リストからサッと選べる。
- **Material（マテリアル・素材）**：木、金属、ガラス、コンクリートなど、表面の材質質感をリアルに変えられる。
- **Transparency（透明度）**：0 にすると普通のブロック、1 にすると完全に透明で見えない空気ブロックになる。

- **半透明のガラスや水も作れる**：数値を「0.5」などにすると、向こう側が透けて見えるガラスや氷が作れる。
- **Reflectance（反射・ツヤ）**：数値を上げると、鏡や磨かれた金属のように周りの空や光をピカピカ反射する。
- **CastShadow（影をつけるか）**：チェックを外すと、そのブロックが地面に落とす影をすっきりと消すことができる。
- **Neon（光る素材）**：マテリアルを「Neon」にすると、看板やビームサーベルのようにピカーッと発光する。
- **材質の細かな模様（Variant）**：慣れてくると、オリジナルの壁紙やタイルの凸凹模様を貼り付けることもできる。
- **色と素材の組み合わせが無限**：「赤い木」や「青く光る金属」など、自由な組み合わせで世界観を作れる。

3. 大きさと向き（トランスフォーム）

ブロックの背の高さ、太さ、傾きを数字でキッチリ合わせる項目です。

- **Size（サイズ・大きさ）**：横幅（X）、高さ（Y）、奥行き（Z）の 3 つの数字で大きさを決める。

- **1 マス単位で細かく揃えられる**：手で引っ張るよりも、数字を「4, 1, 2」のように直接打つほうが綺麗に作れる。
- **Position（ポジション・位置）**：ワールドのどこにあるかを表す「住所の番号」のような 3 つの数字。
- **床から浮かせたり沈めたり**：真ん中の数字（Y）を大きくすると上空へ浮き、小さくすると地面へ近づく。
- **Orientation（向き・角度）**：ブロックがどの方角を向いているかを「0 度〜360 度」の数字で指定する。
- **坂道や階段を作る**：角度の数字を「45」などに変えるだけで、綺麗な滑り台やスロープが作れる。
- **マウス操作より狂いが出ない**：手動で動かしてズレてしまった隙間も、位置の数字を揃えればピタッとくっつく。
- **Pivot（ピボット・回転の中心軸）**：ドアのように「端っこを中心に回転させたい」時の軸の位置を決められる。
- **マイナスの数字も使える**：位置や角度は、マイナス（-）の数字を入れることで反対方向へ動かせる。

- **3 つの数字の並び順は「横、縦、奥行き」**：基本的にすべて（X, Y, Z）の順番で並んでいると覚えると分かりやすい。

4. 物理とぶつかり判定（コリジョン）

壁として通り抜けられないようにするか、お化けのように通り抜けられるかを決めます。

- **CanCollide（通り抜けるかどうか）**：チェックを外すと、ブロックに体が触れずにすり抜けられるようになる。
- **隠し通路やワナが作れる**：見た目は壁なのに通り抜けられる「隠し扉」がチェック 1 つで作れる。
- **水やホログラムの表現に必須**：海や煙のように「中に入れる物体」を作るときは、このチェックを外しておく。
- **CanTouch（触れたイベントを起こすか）**：踏んだら爆発する床や、触れたらコインがもらえる仕掛けで使うスイッチ。
- **CanQuery（センサーに反応するか）**：レーザー光線や AI の視線感知がこのブロックを無視するかどうかを決める。
- **Massless（重さをなくす）**：帽子やアクセサリーを付けたとき、重さでキャラクターの動きが重くならないようにする。

- **CustomPhysicalProperties（滑りやすさ）**：ツルツル滑るツルピカの氷の床や、全然滑らないゴムの床を細かく作れる。
- **Friction（摩擦・ブレーキ力）**：数値を下げるとソリのように滑り、上げると坂道でもピタッと止まれる。
- **Elasticity（跳ね返り・弾力）**：スーパーボールのように、落としたときにどれくらいボヨンと跳ねるかを決められる。
- **空中ブロックの落下注意**：通り抜け設定と後述の「アンカー」を間違えると、ブロックが奈落の底へ消えていく原因になる。

5. アンカー（空中に浮かせて固定する超重要設定）

初心者が一番最初につまずきやすい「ブロックが崩れて落ちる問題」を解決する設定です。

- **Anchored（アンカー・固定スイッチ）**：チェックを入れると、重力を無視してその場所にピタッとネジ止めされる。
- **チェックがないと全部落ちる**：チェックを入れ忘れたブロックは、ゲーム開始と同時に重力でガラガラと崩れ落ちる。
- **浮かぶ島や足場が作れる**：空中にブロックを置き、アンカーにチェックを入れるだけで絶対に落ちない足場になる。

- **初心者のトラブル第１位**：「作った建物が全部崩れた！」という時は、100%このチェックが入っていないことが原因。
- **押してもビクともしなくなる**：プレイヤーが体当たりしても、車がぶつかっても、アンカーされた物は１ミリも動かない。
- **ドミノや岩を作るときは外す**：倒したいドミノや、坂道を転がしたい大岩などは、わざとこのチェックを外しておく。
- **ゲームが軽くなる**：アンカーを入れて固定しておくと、パソコンが「落ちる計算」をしなくて済むため動作が軽くなる。
- **まとめて一気にオンにできる**：複数のブロックをまとめて選んでからチェックを入れれば、一括で固定できる。
- **動く乗り物には付けない**：自分で運転する車や船を作るときは、動かなくなってしまうのでチェックを外す。
- **「固定ピン留めボタン」と覚える**：コルクボードにピンで止めるように、空間に留めるための命綱スイッチ。

6. 名前と整理（探しやすいように名札をつける）

どれが何のパーツなのかを整理して迷子を防ぐための項目です。

- **Name（名前）**：初期状態では全部「Part」になっているので、わかりやすい名前に書き換えられる。
- **日本語も使えるが英語推奨**：プログラムから動かしたい物は「Door」「Floor」など半角英数字で名付けるのが安全。
- **大文字と小文字は区別される**：「Apple」と「apple」は別の物として扱われるので注意が必要。
- **Parent（親・所属している場所）**：そのブロックがどのフォルダやモデルの中に入っているかを表示する。
- **ClassName（種類・種別）**：これは書き換えられない名札で、「私は普通のパーツです」「私は光です」という正体を表す。
- **同じ名前が何個あっても大丈夫**：コインを 100 個置いて全部名前に「Coin」と付けてもエラーにはならない。
- **プログラムから探す目印になる**：AI やスクリプトに「Door という名前の壁を開けて」と指示するために名前を使う。
- **変な記号は避ける**：スペース（空白）やスラッシュ（/）を入れすぎると、後でプログラムから指定しづらくなる。

- **完成した物はグループ化して命名**：壁や屋根をまとめたら、全体の名前を「House」に変えるとスッキリ片付く。
- **こまめな命名が未来の自分を救う**：パーツが1000個を超えたとき、全部「Part」のままだと探すのが地獄になる。

7. ライト・光の設定（部屋を明るく照らす小道具）

ブロックの中に「明かり（PointLightなど）」を入れた時に現れる特別な設定です。

- **Brightness（明るさの強さ）**：電球のワット数を上げるように、光の眩しさを数字で強くできる。
- **Color（光の色）**：白い蛍光灯、温かみのあるオレンジ色のランプ、怪しい紫の光など自由に変えられる。
- **Range（照らす範囲・届く距離）**：光がどれくらい遠くまで届くかをメーター単位の数字で広げられる。
- **Shadows（光の影を落とすか）**：チェックを入れると、家具や柱の後ろにリアルな影がクッキリ生まれる。
- **Enabled（オン・オフのスイッチ）**：チェックを外すと明かりがフッと消える（停電や懐中電灯の消灯に使える）。

- **PointLight（電球タイプ）**：自分を中心に、まあるく全方位へふわっと光を広げる明かり。
- **SpotLight（懐中電灯タイプ）**：ステージのスポットライトのように、決めた 1 方向だけを丸く照らす光。
- **Angle（スポットライトの角度）**：光のメガホンを広げるように、照らす扇形の幅を広くしたり狭くしたりできる。
- **SurfaceLight（蛍光灯パネルタイプ）**：ブロックの「平らな一面」から前へ向かって面全体で光を放つ。
- **使いすぎ注意**：影を落とすライトを何十個も置きすぎると、ゲーム画面がカクカク重くなる原因になる。

8. 文字や画像の設定（看板・テクスチャ・デカール）

壁にポスターを貼ったり、立体看板に文字を書いたりした時にいじる項目です。

- **Texture（画像のアセット ID）**：アップロードしたイラストや写真の番号を入れると、壁に画像がペタッと貼られる。
- **Face（画像を貼る面）**：ブロックの「前・後ろ・上・下・右・左」のどの面に画像を貼るかを指定する。

- **Color3（画像の色合い乗せ）**：貼った画像全体に赤や青のカラーフィルターを上からかけることができる。
- **Text（看板に書く文字）**：3D テキストや看板に表示させたいメッセージをキーボードで打ち込む。
- **TextScaled（文字の自動サイズ調整）**：チェックを入れると、枠の大きさに合わせて文字が最大まで自動でデカくなる。
- **Font（フォント・文字のフォント）**：可愛いポップ体、かっこいい SF 風、ホラーな手書き風など文字の形を選べる。
- **TextColor3（文字自体の色）**：黒板のチョークのように、文字の色だけを好きな色に変えられる。
- **TextStrokeTransparency（文字のフチ取り）**：文字の外側に黒いフチ取り線をつけて、背景が明るくても読みやすくする。
- **StudsPerTile（テクスチャの繰り返し）**：レンガの画像を何マスごとに細かく繰り返して並べるかを調整する。
- **Transparency（ステッカーの透明化）**：お化けのイラストなどをうっすら透けさせて表示させたいときに数値をいじる。

9. 2D 画面ボタンの設定（UI・画面のメーター）

スマホや PC の画面上にペタッと張り付くボタンや体力バーの設定です。

- **Size（画面上の大きさ）**：ピクセル数や「画面全体の何パーセントか」という割合で大きさを決める。
- **Position（画面上の配置場所）**：画面の左上、ど真ん中、右下など、ボタンを置く位置を指定する。
- **AnchorPoint（ボタンを引っ掛ける画鋲の位置）**：ボタンの「左上端」基準で置くか、「ど真ん中」基準で置くかを決める。
- **BackgroundColor3（枠やボタンの下地の色）**：ボタンの背景の色をパレットから選んで塗りつぶす。
- **BackgroundTransparency（背景の透明化）**：1 にすると下地が消えて、文字やアイコンだけが画面に浮き出る。
- **AutoButtonColor（押し心地の演出）**：チェックが入っていると、プレイヤーが押した瞬間にボタンが少し暗くなって「押した感」が出る。
- **Visible（画面に出すか隠すか）**：チェックを外すと画面からフッと消える（カバンを閉じた状態などに使う）。

- **ZIndex（重なり順の優先度）**：数字が大きいボタンほど手前に表示され、後ろのパネルに隠れなくなる。
- **CornerRadius（角を丸める設定）**：角丸パーツ（UICorner）を入れると、四角いボタンをスマホ風の丸いボタンに変えられる。
- **Active（後ろを触らせないガード）**：チェックを入れると、ボタンを押した時に後ろのゲーム画面をクリックしなくなる。

10. 人間・アバターの設定（歩く速さやジャンプ力）

プレイヤーや動くキャラクター（Humanoid）の身体能力を決める特別な項目です。

- **WalkSpeed（歩くスピード・足の速さ）**：基本は「16」。数字を 50 にすると猛ダッシュ、5 にするとノロノロ歩きになる。
- **JumpPower / JumpHeight（ジャンプの高さ）**：高く跳ばせてビルを飛び越えさせたり、0 にしてジャンプ禁止にできる。
- **Health（現在の体力・HP）**：ダメージを受けると減る数字。0 になるとバラバラになって倒れる。
- **MaxHealth（最大の体力上限）**：体力の最大値。基本は「100」だが、ボス敵なら「10000」などに増やせる。

- **DisplayName（頭の上に浮かぶ名前）**：キャラクターの頭の上に白文字で表示される名前を自由に変えられる。
- **AutoRotate（自動で進行方向を向くか）**：歩いた方向に体をくるっと向けるかどうかのスイッチ。
- **UseJumpPower（ジャンプの計算方式）**：旧来の「パワー計算」にするか、最新の「跳ぶ高さ（メートル計算）」にするかを選ぶ。
- **PlatformStand（気絶・ぐったり状態）**：チェックを入れると足の力が抜けて、ラグドール（ぐったり人形）のように倒れ込む。
- **Sit（座るフラグ）**：チェックを入れると、椅子がなくてもその場でストンとお尻をつけて座る。
- **HipHeight（腰の高さ・浮き沈み）**：地面から腰までの高さを調節して、足が地面に埋まったり宙に浮いたりするのを防ぐ。